Multi-Layer Perceptrons (MLP) & Stochastic Gradient Descent (SGD)

Kevin Bryson

Images acknowledged from:

Deep Learning with PyTorch, Manning Publishers

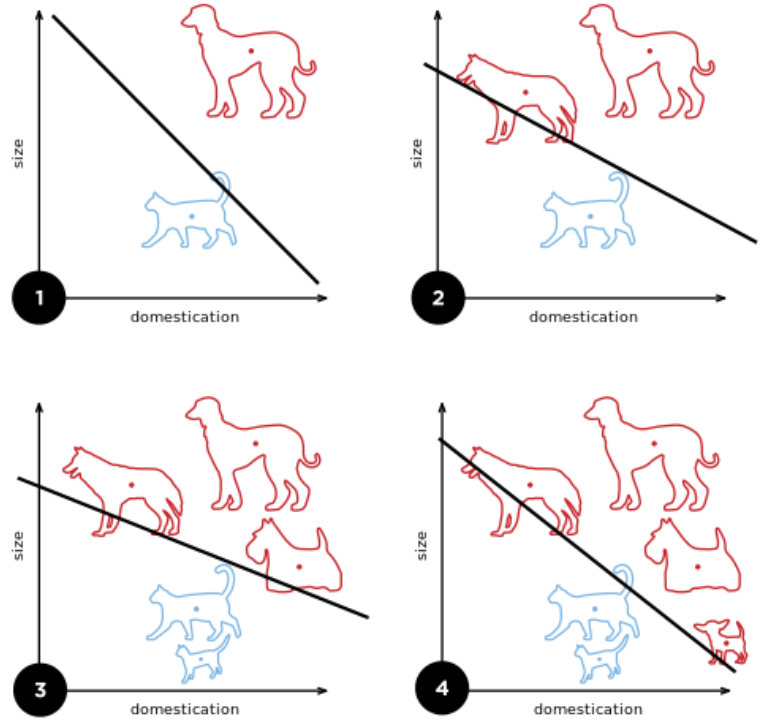
Overview

- We will first look at the idea of using gradient descent to improve a very simple network consisting of a single neuron.

- We will introduce the key components of:
  - The model f(**x**; **w**)
  - The cost or loss function L(f(**x**; **w**), y)
  - Optimizing the weights / bias using stochastic gradient descent.

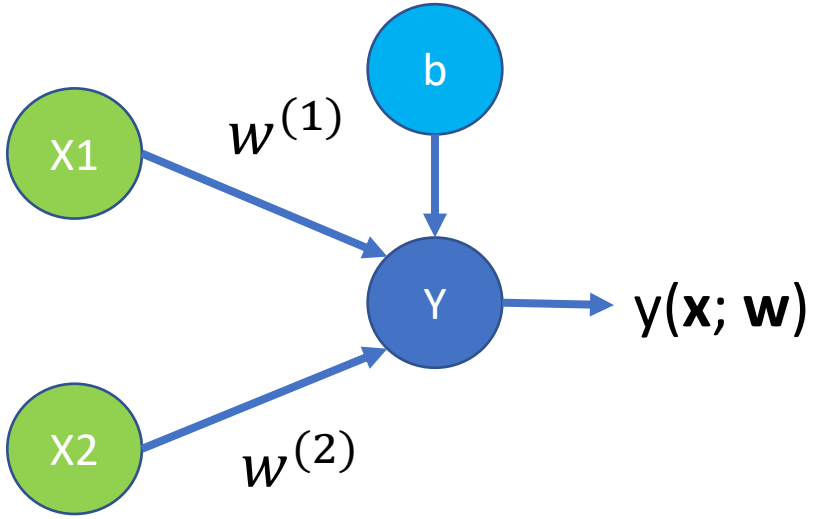
- Then we will show how this applies to neural networks consisting of multiple layers of functions that map vectors to vectors in a non-linear manner.
- We will look at different types of activation function and why these are required.
- Finally, you should gain an overall understanding how gradient descent can be used to optimize weight and bias parameters in deep neural networks to minimize loss over a given training data.

You should be familiar with this …

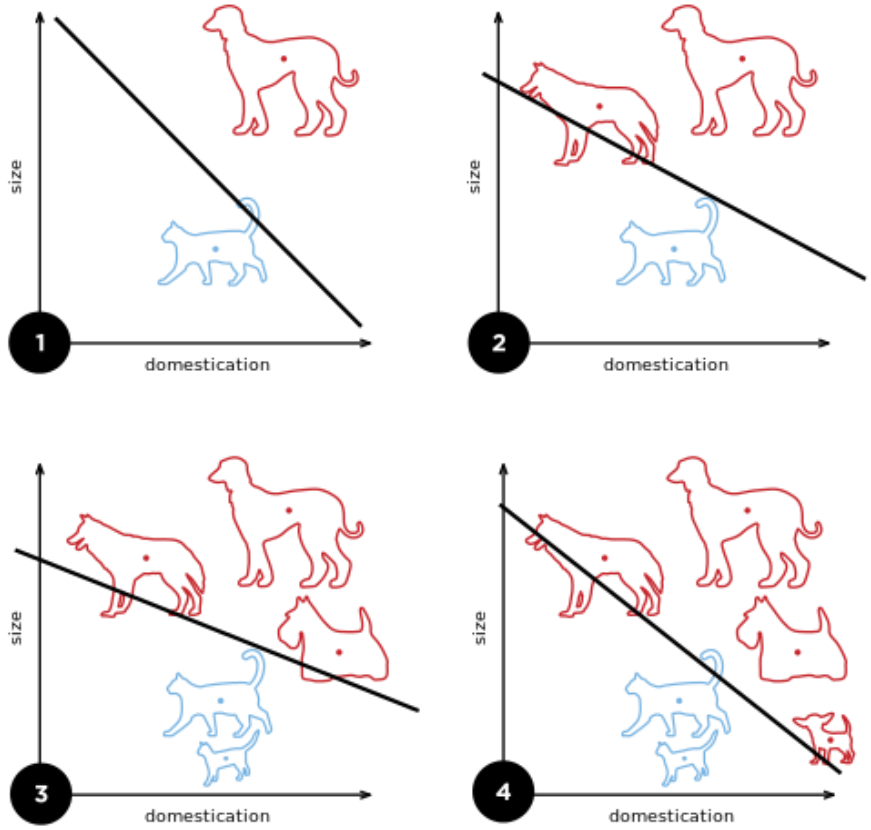


https://en.wikipedia.org/wiki/Perceptron

A single perceptron neuron with 2 inputs (Rosenblatt, 1958)

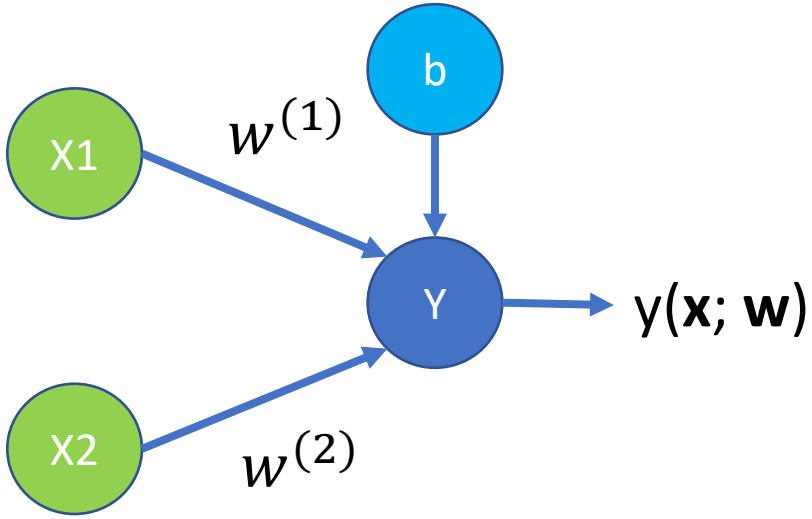


$$y(\mathbf{x}) = \phi\left(b + \sum_i w_i x_i\right)$$



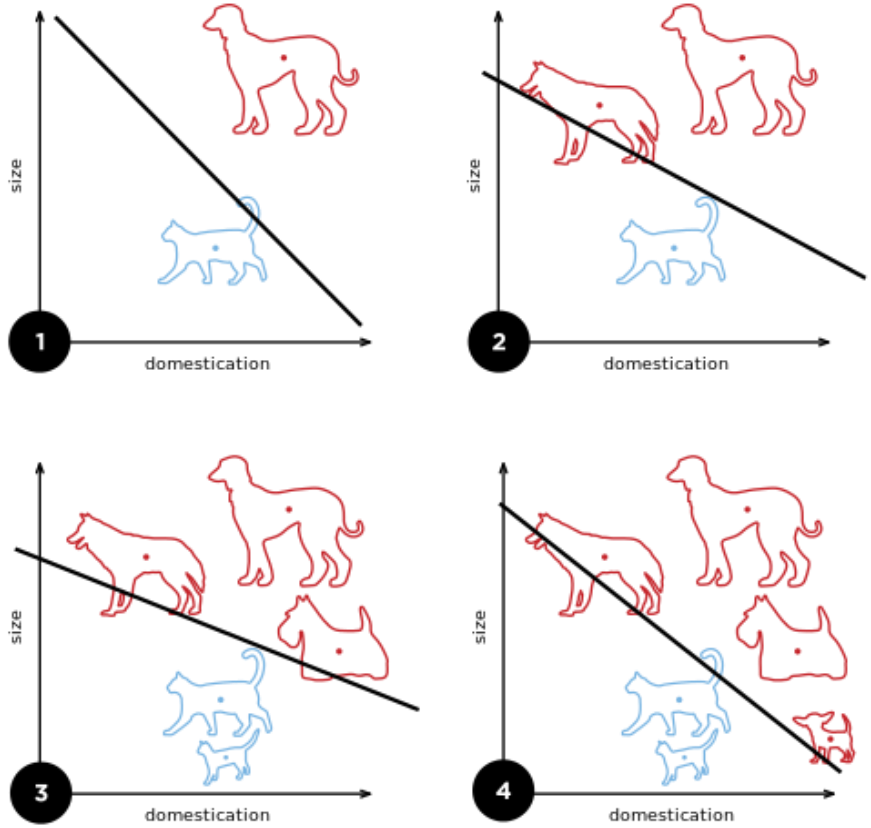
https://en.wikipedia.org/wiki/Perceptron

Introduce a cost or loss function to be optimized … is MSE appropriate for this?



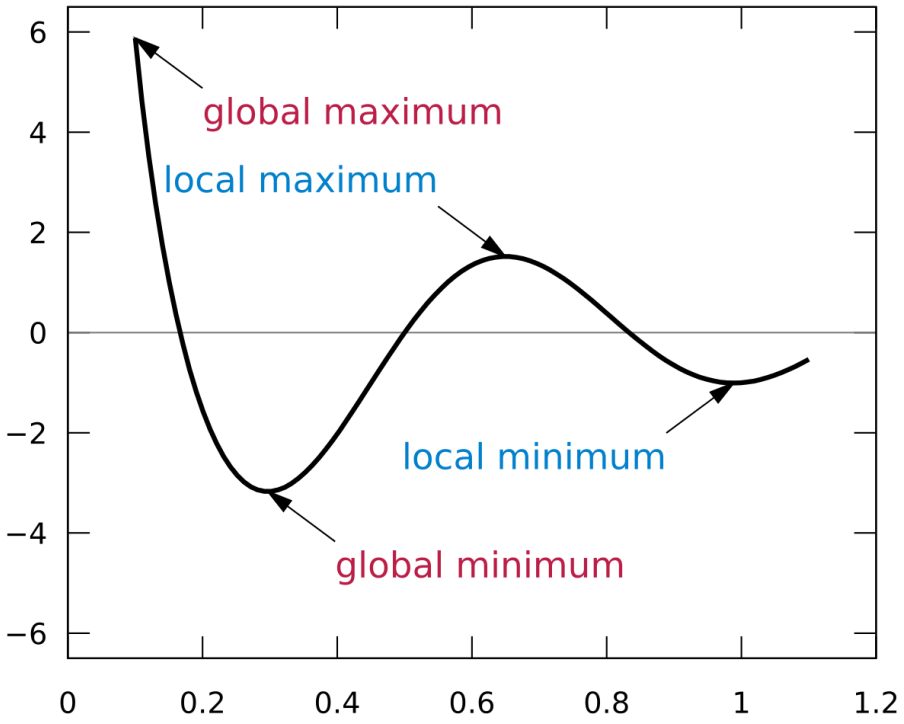
$$y(\mathbf{x}) = \phi\left(b + \sum_i w_i x_i\right)$$

$$L(\mathbf{x}, t) = \frac{1}{2}(y(\mathbf{x}) - t)^2$$



https://en.wikipedia.org/wiki/Perceptron

So learning is optimisation

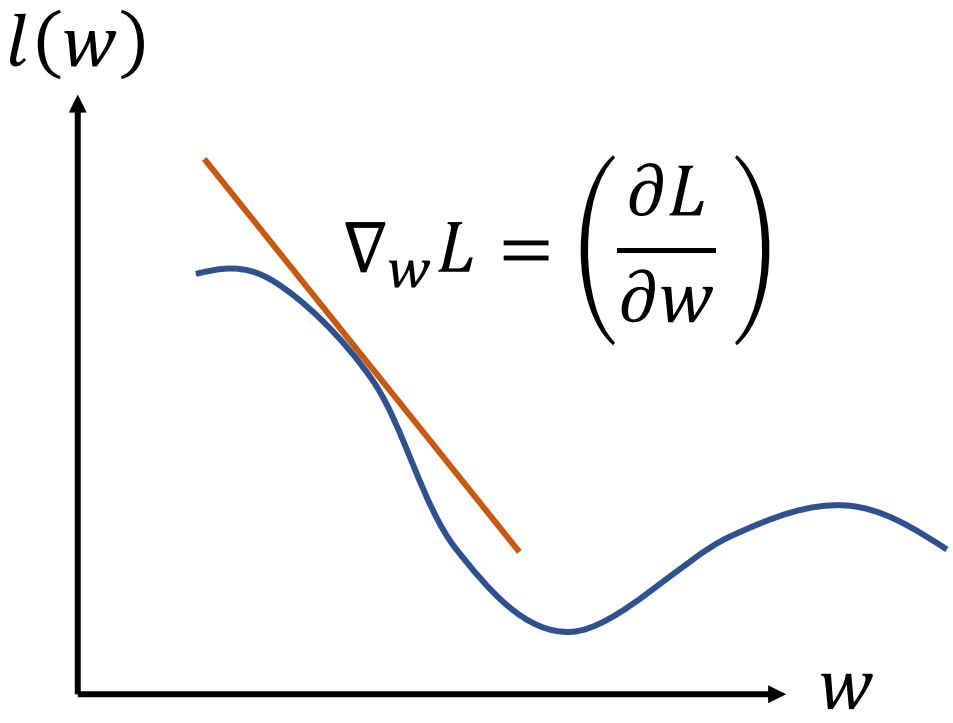


- If your function is **convex**, you may have efficient algorithms for finding the **global minimum** (quadratic optimisation: SVM)
- If not, you are left with iterative algorithms only guaranteed to give you a **local optimum**.
- One of the simplest and most popular is **gradient descent**

https://en.wikipedia.org/wiki/Maxima_and_minima#/media/File:Extrema_example_original.svg

Optimizing the cost or loss based on a single parameter

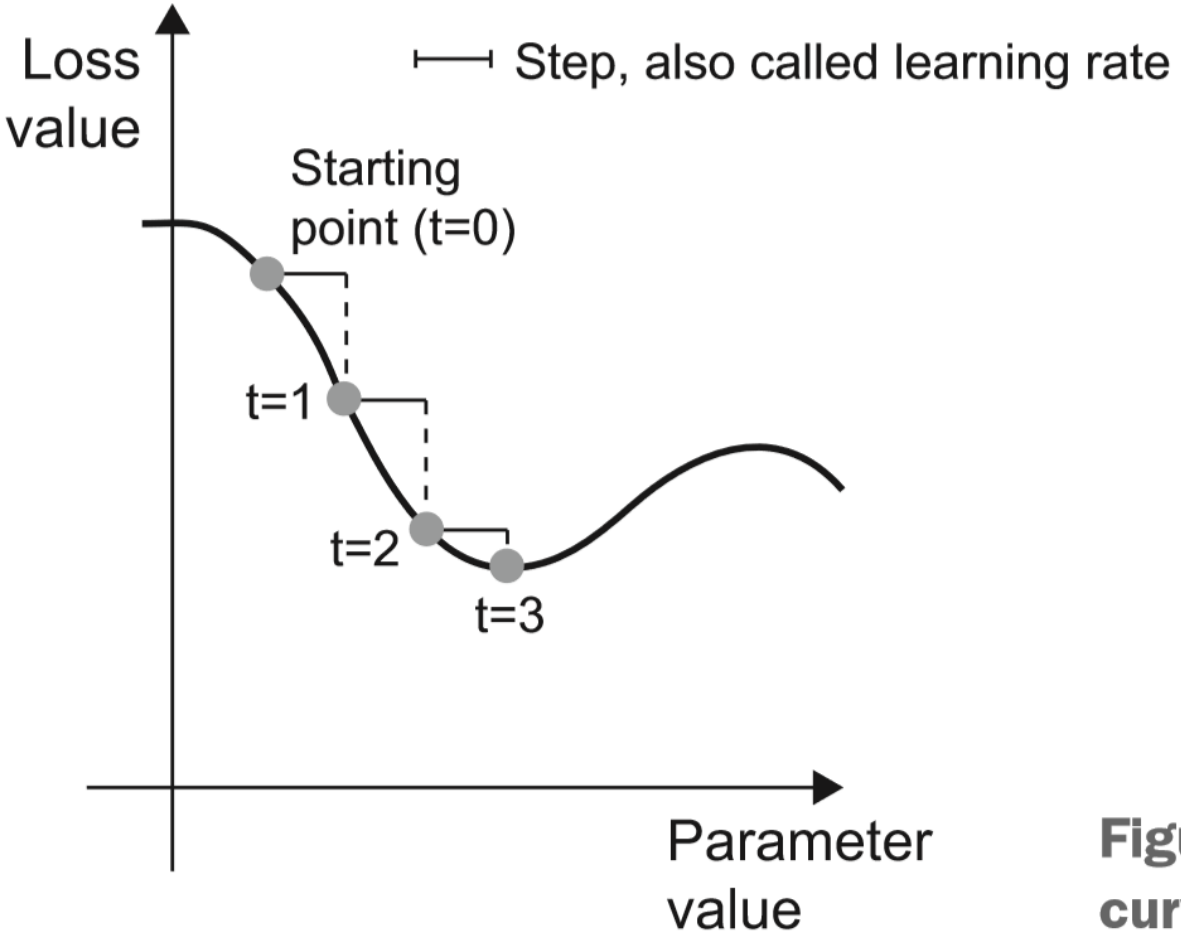
Which way should we change the weights?



$$l(w) = L[f(x; w), y]$$

This requires the model $f(x; w)$ and the loss function to be smooth.

Gradient descent



$$w = w - \alpha \nabla_w L$$

Where α is learning rate

Figure 2.11 SGD down a 1D loss curve (one learnable parameter)

Step size or learning rate is important….

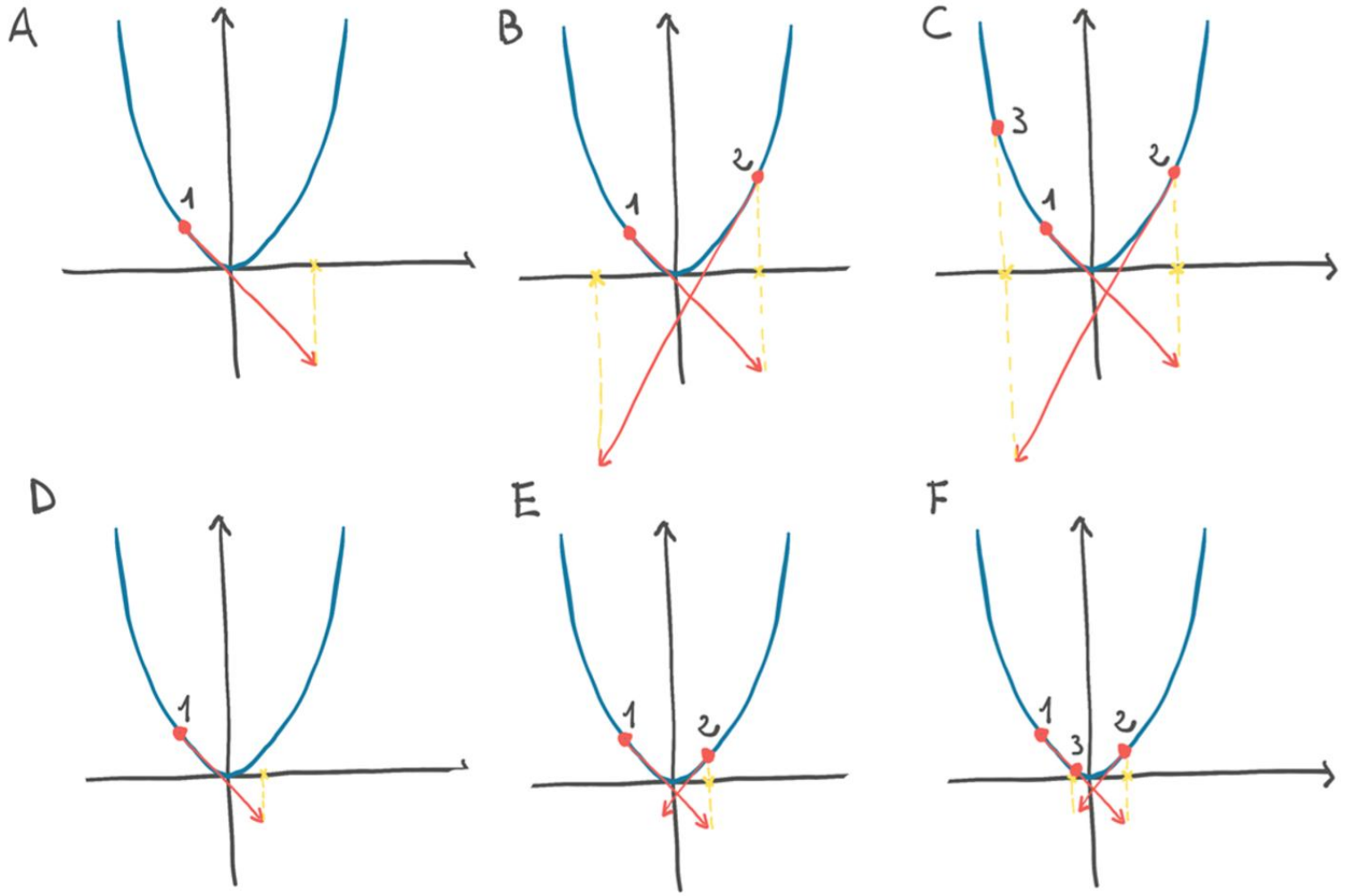
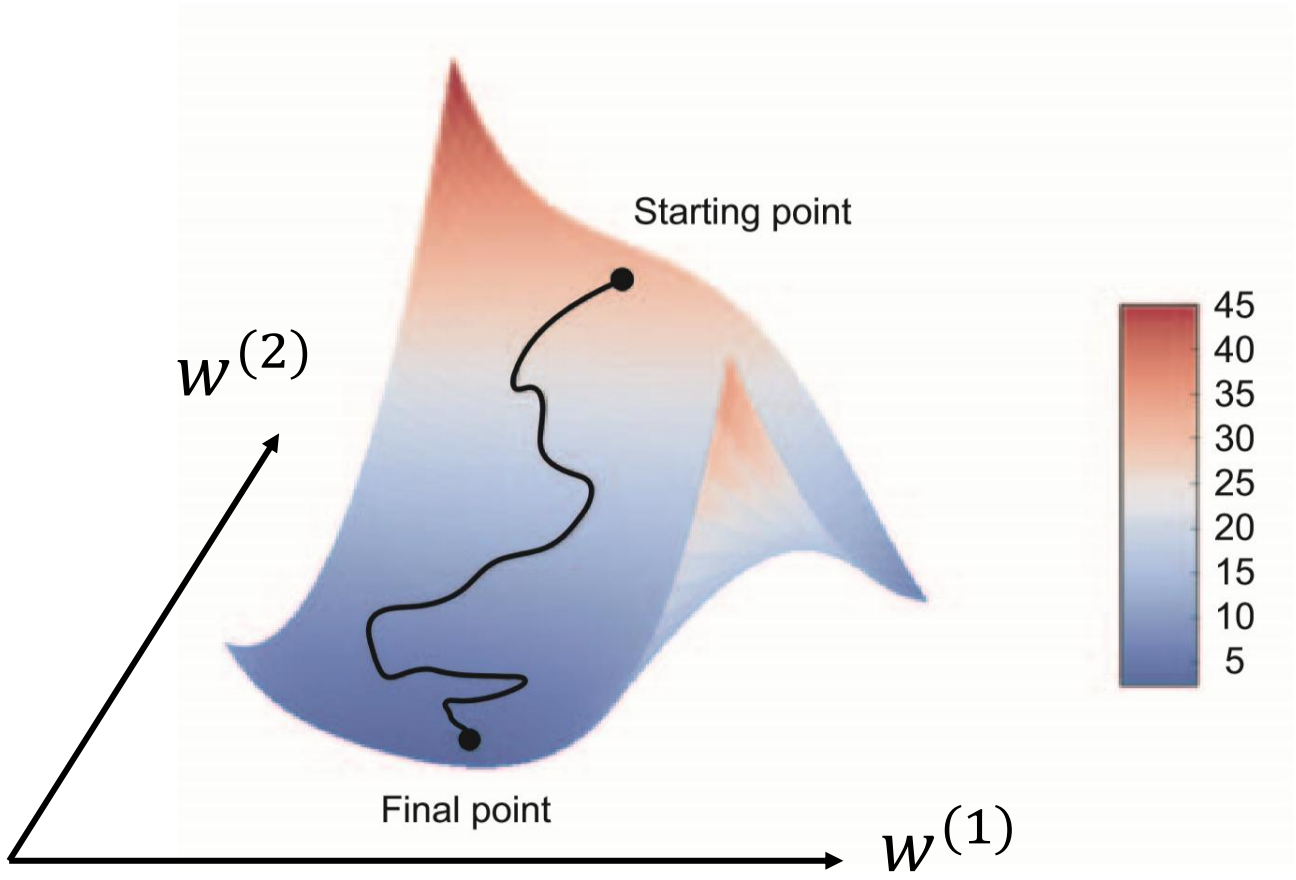


Figure 5.7 TOP: diverging optimization on convex function (parabola-like) due to large steps; BOTTOM: converging optimization with small steps.

Gradient descent of loss which depends on a vector of weight parameters



$$l(w) = L[f(x; \boldsymbol{w}), y]$$

$$\nabla_{\boldsymbol{w}} L = \left(\frac{\partial L}{\partial w^{(1)}}, \frac{\partial L}{\partial w^{(2)}} \right)$$

$$= \left(\frac{\partial L}{\partial f} \cdot \frac{\partial f}{\partial w^{(1)}}, \frac{\partial L}{\partial f} \cdot \frac{\partial f}{\partial w^{(2)}} \right)$$

$$\boldsymbol{w} = \boldsymbol{w} - \alpha \nabla_w L$$

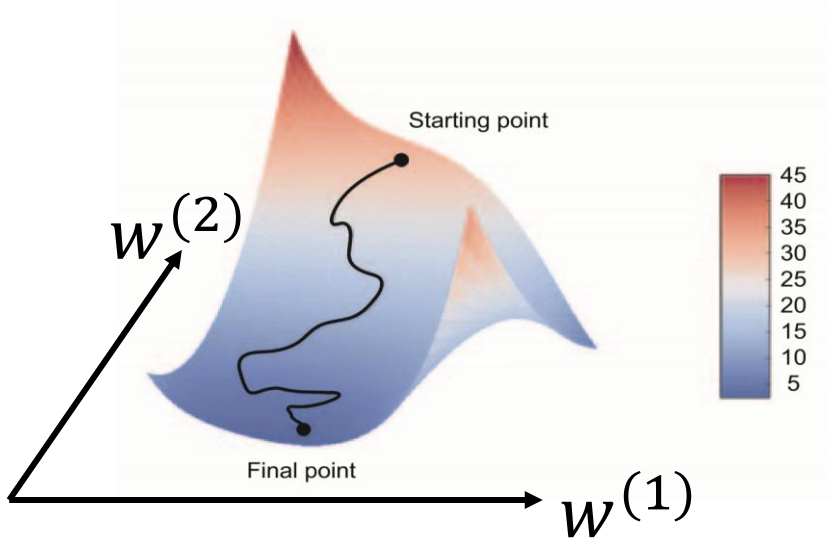
Figure 2.12 Gradient descent down a 2D loss surface (two learnable parameters)

How should the gradient be calculated when we have a thousand (**x**, y) training points?

- In theory, all thousand (**x**, y) training should be used to determine the gradient at each point on the loss(**w**) surface by cycling through all of them and averaging the gradient.

- But this is very inefficient !

- **Stochastic Gradient Descent (SGD)** calculates the average gradient using a **mini-batch** of samples ... say 100.



- This makes the downhill walk more stochastic ... hence the name.

SGD uses approximate gradients … but this can be beneficial to avoid local minima !

Also, this 2D view is misleading since, in high dimensional spaces, ‘stationary points’ (gradient = 0) are likely to be saddle points where the search point can escape along one of the dimensions where the function is decreasing.

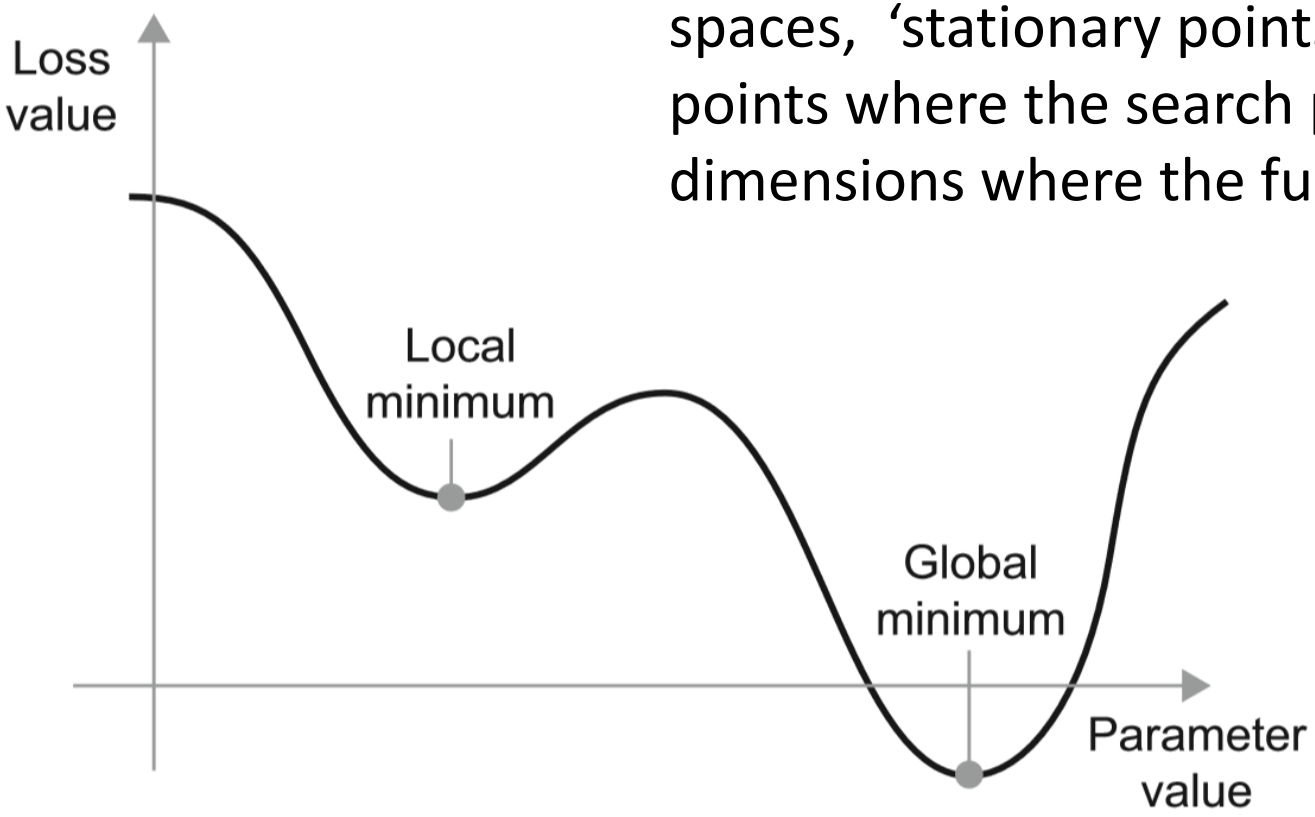
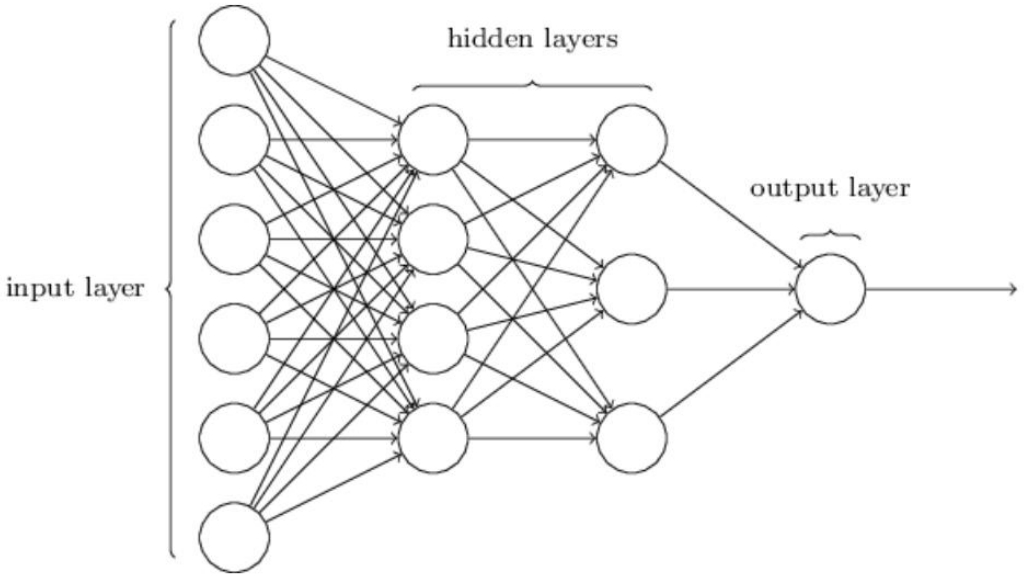
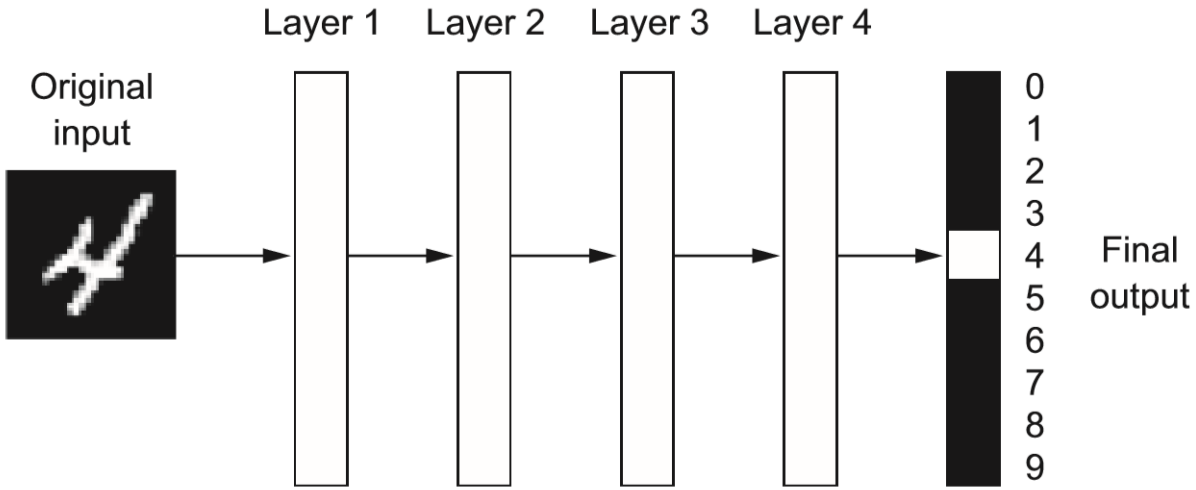


Figure 2.13 A local minimum and a global minimum

Applying this to neural nets

Simple deep networks



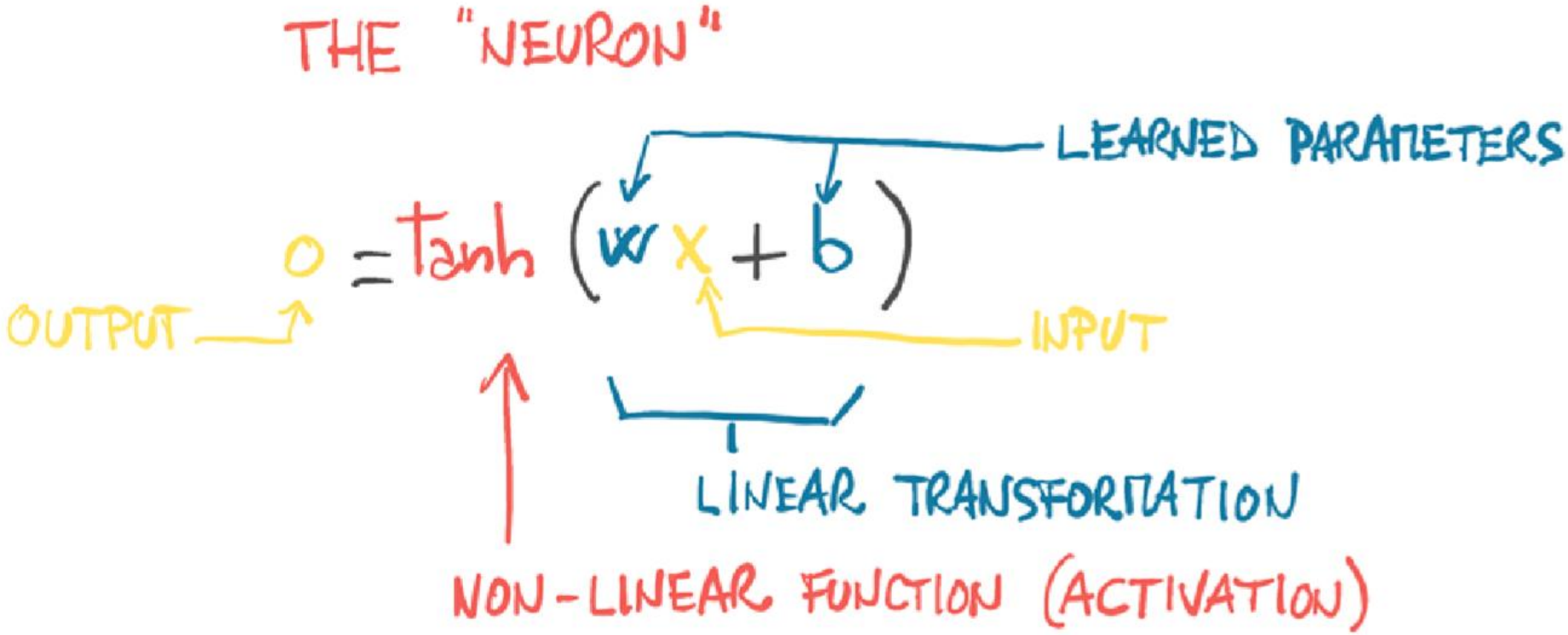


The output y can be calculated as a function of the input vector x:

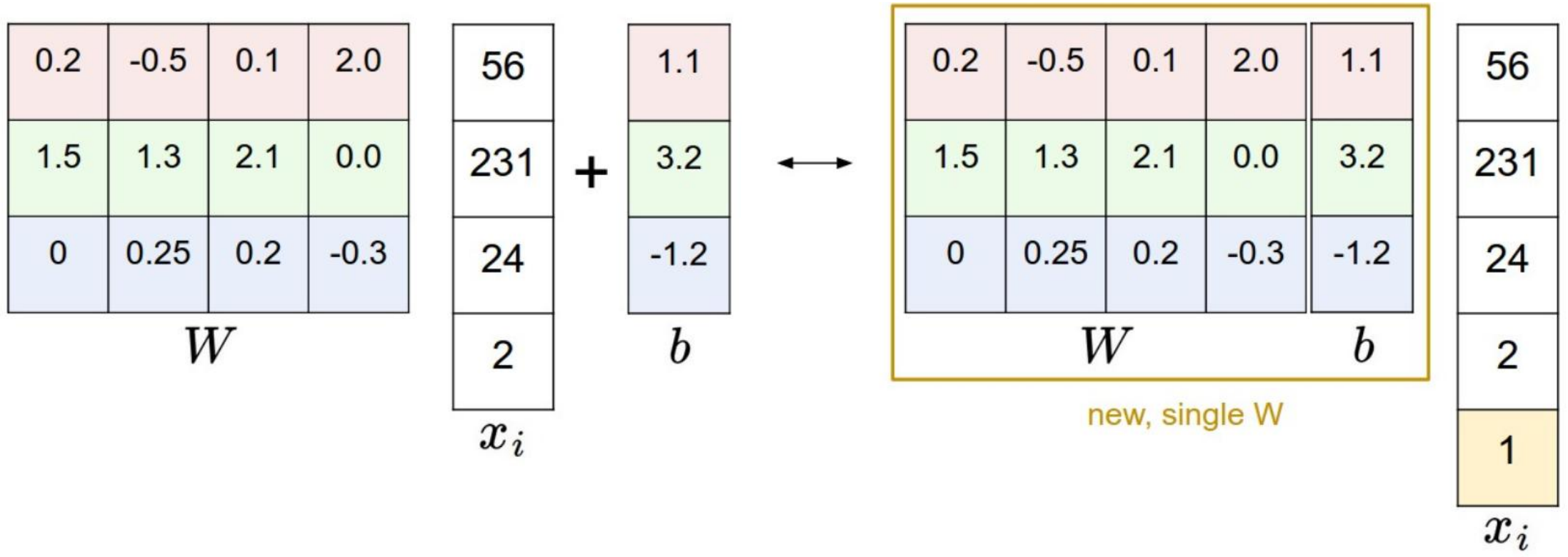
$$y = f_4(f_3(f_2(f_1(x)))),$$

where $f_1(x) = g_1(W_1 x)$, where $g_1()$ is a nonlinear *activation function* and W_1 is the weight matrix of the first layer of the network.

(Chollet, 2018)


THE "NEURON"
LEARNED PARAMETERS
o = tanh (wx + b)
OUTPUT
INPUT
LINEAR TRANSFORMATION
NON-LINEAR FUNCTION (ACTIVATION)

Including the bias in the weight matrix



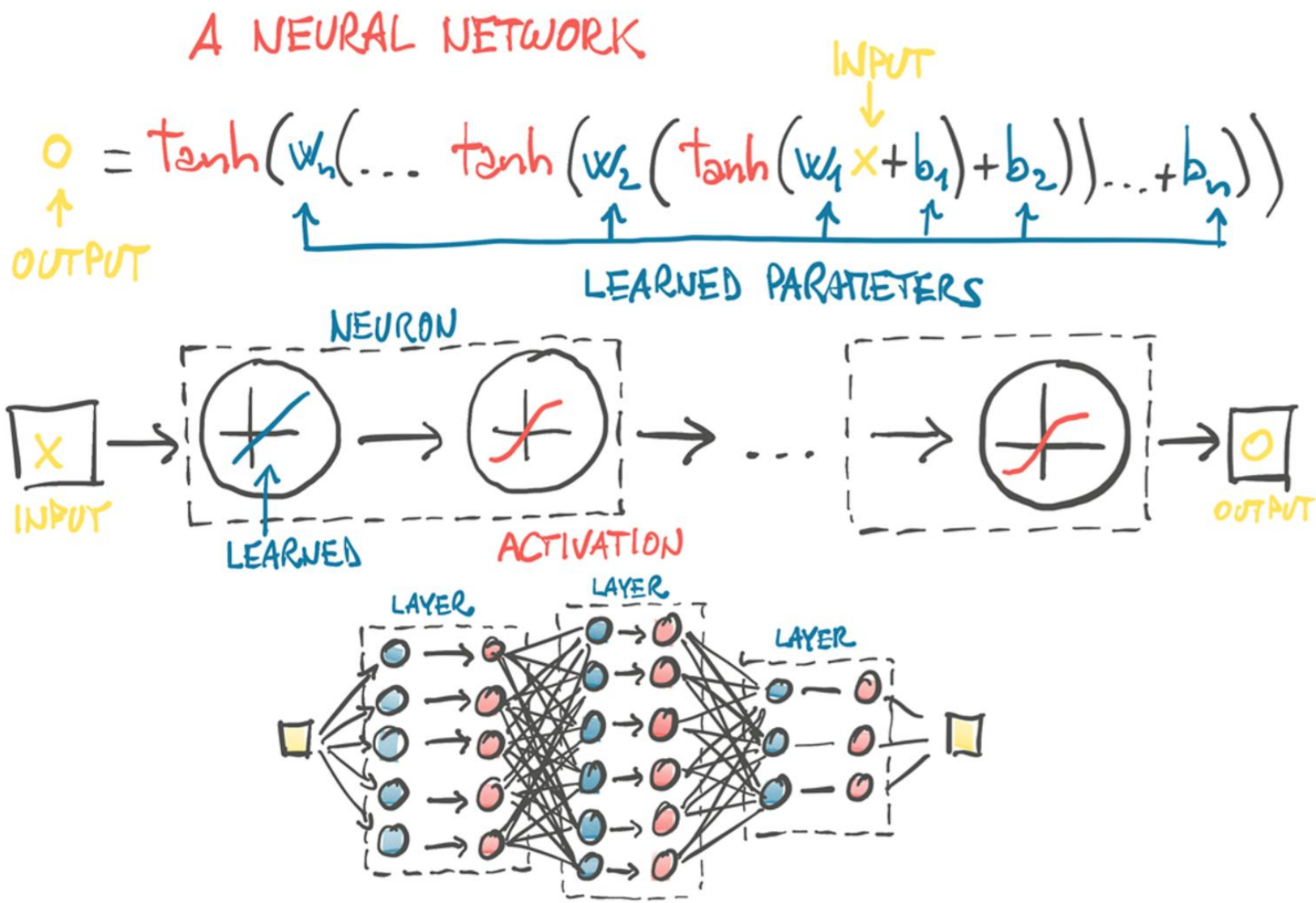
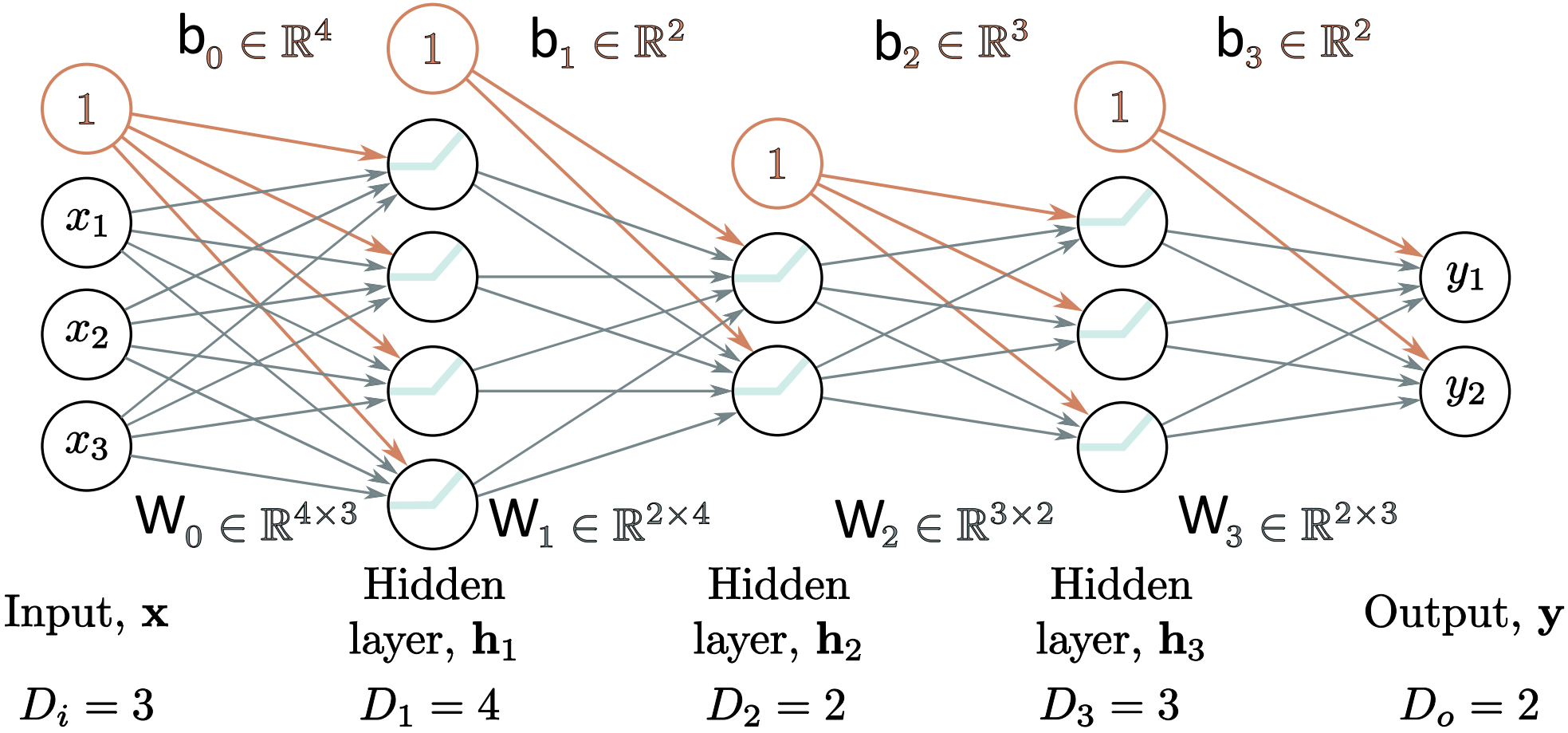


Figure 6.3 A neural network with three layers.

Another Example



Modified from: Understanding Deep Learning (Prince, 2025)

General equations for deep network

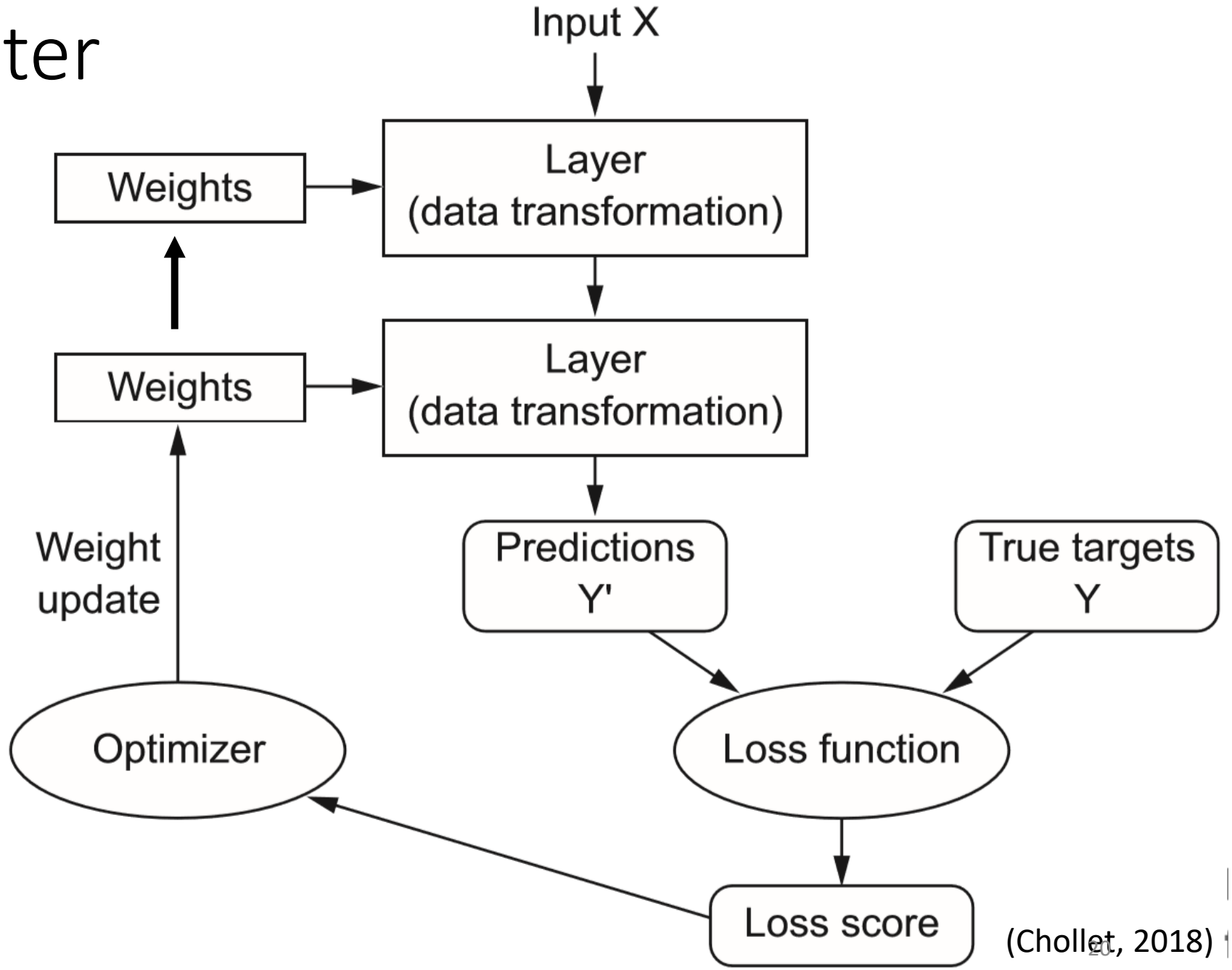
$$
\begin{aligned}
\mathbf{h}_1 &= \mathbf{a}[\mathsf{b}_0 + \mathsf{W}_0\mathbf{x}] \\
\mathbf{h}_2 &= \mathbf{a}[\mathsf{b}_1 + \mathsf{W}_1\mathbf{h}_1] \\
\mathbf{h}_3 &= \mathbf{a}[\mathsf{b}_2 + \mathsf{W}_2\mathbf{h}_2] \\
&\vdots \\
\mathbf{h}_K &= \mathbf{a}[\mathsf{b}_{K-1} + \mathsf{W}_{K-1}\mathbf{h}_{K-1}] \\
\mathbf{y} &= \mathsf{b}_K + \mathsf{W}_K\mathbf{h}_K,
\end{aligned}
$$

$$
\mathbf{y} = \mathsf{b}_K + \mathsf{W}_K\mathbf{a}\left[\mathsf{b}_{K-1} + \mathsf{W}_{K-1}\mathbf{a}\left[\ldots\mathsf{b}_2 + \mathsf{W}_2\mathbf{a}\left[\mathsf{b}_1 + \mathsf{W}_1\mathbf{a}\left[\mathsf{b}_0 + \mathsf{W}_0\mathbf{x}\right]\right]\ldots\right]\right]
$$

Modified from: Understanding Deep Learning (Prince, 2025)

Overall parameter optimization

All weights updated at the same time …



(Chollet, 2018)

Why do we have activation functions?

If, as before the output y can be calculated as a function of the input vector x:

$$y = f_4(f_3(f_2(f_1(x)))),$$

and we had no nonlinear functions $g_i()$ within each layer, then

$$y = W_4 W_3 W_2 W_1 x,$$

which would be equivalent to a single layer with a weight matrix $W = W_4 W_3 W_2 W_1$, such that the network was:

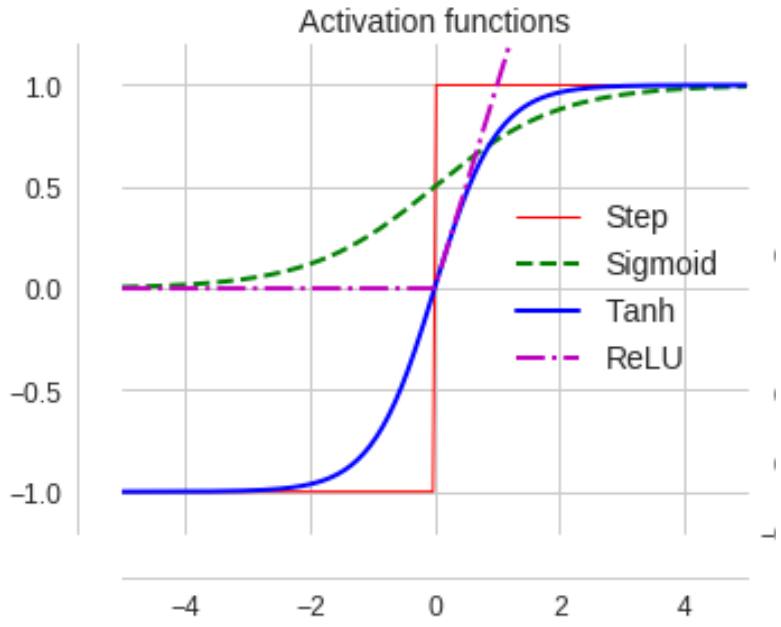
$$y = Wx,$$

so having multiple linear layers has no representational advantage over a single layered network.

Activation functions – step function (Rosenblatt, 1950s)

- A simple step function was used for the original perceptron developed in the 1950s:

$$f(s) = \begin{cases} 1 & \text{If } \mathbf{w}.\mathbf{x} + \mathbf{b} > 0 \\ -1 & \text{otherwise} \end{cases}$$



- Gradient is not defined at 0 … also gradient is zero elsewhere … this makes gradient descent with this activation function difficult !

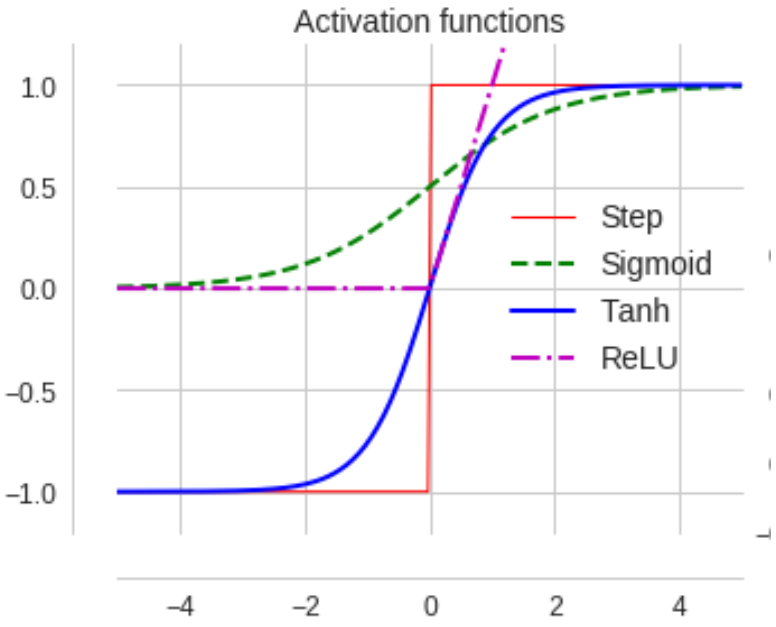
Activation functions – sigmoid (1980s ...)

Individual nodes/neurons in the network have a nonlinear activation function $a = g(z)$, which transforms the sum of weighted inputs (possibly also including a bias term b) from the previous layer, e.g. $z = Wx + b$. The earliest approach was the hard threshold used in the Perceptron.

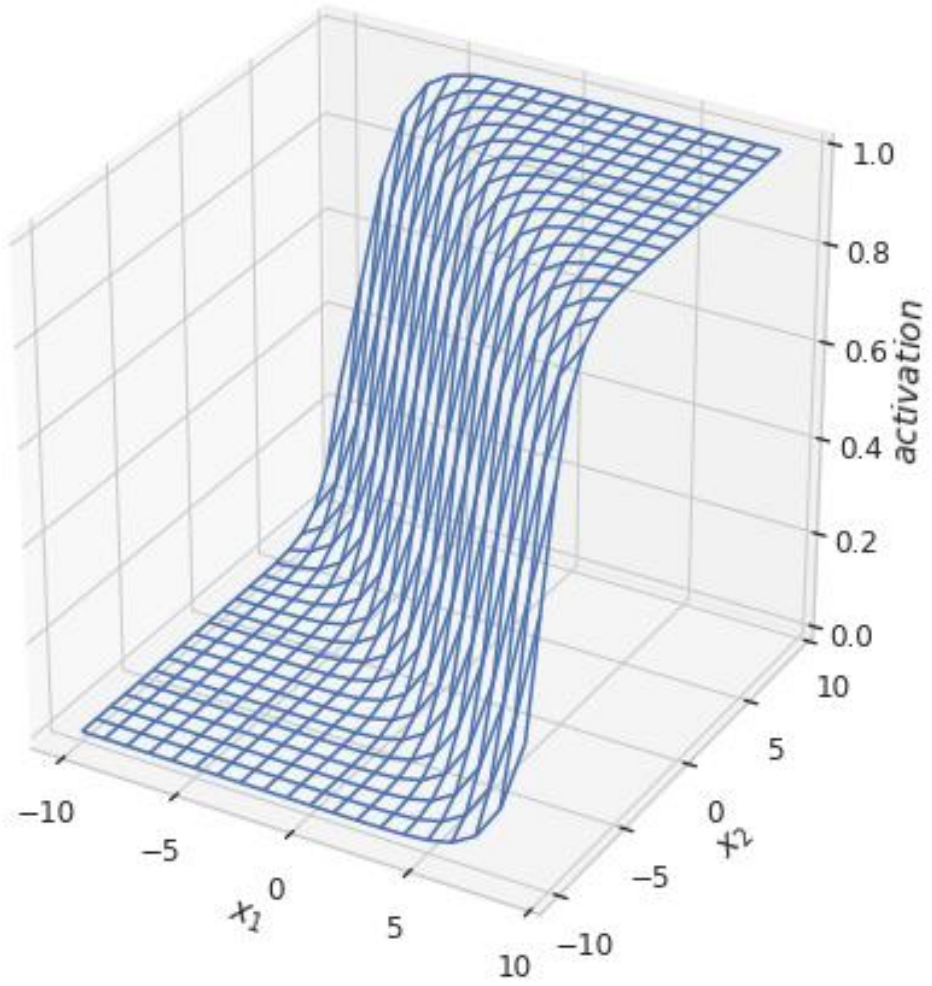
Sigmoid: The *sigmoid function* was then used later on as an approximation to a biological neuron's firing rate.

$$a = \frac{1}{1 + e^{-(Wx+b)}} = \frac{1}{1 + e^{-z}}.$$

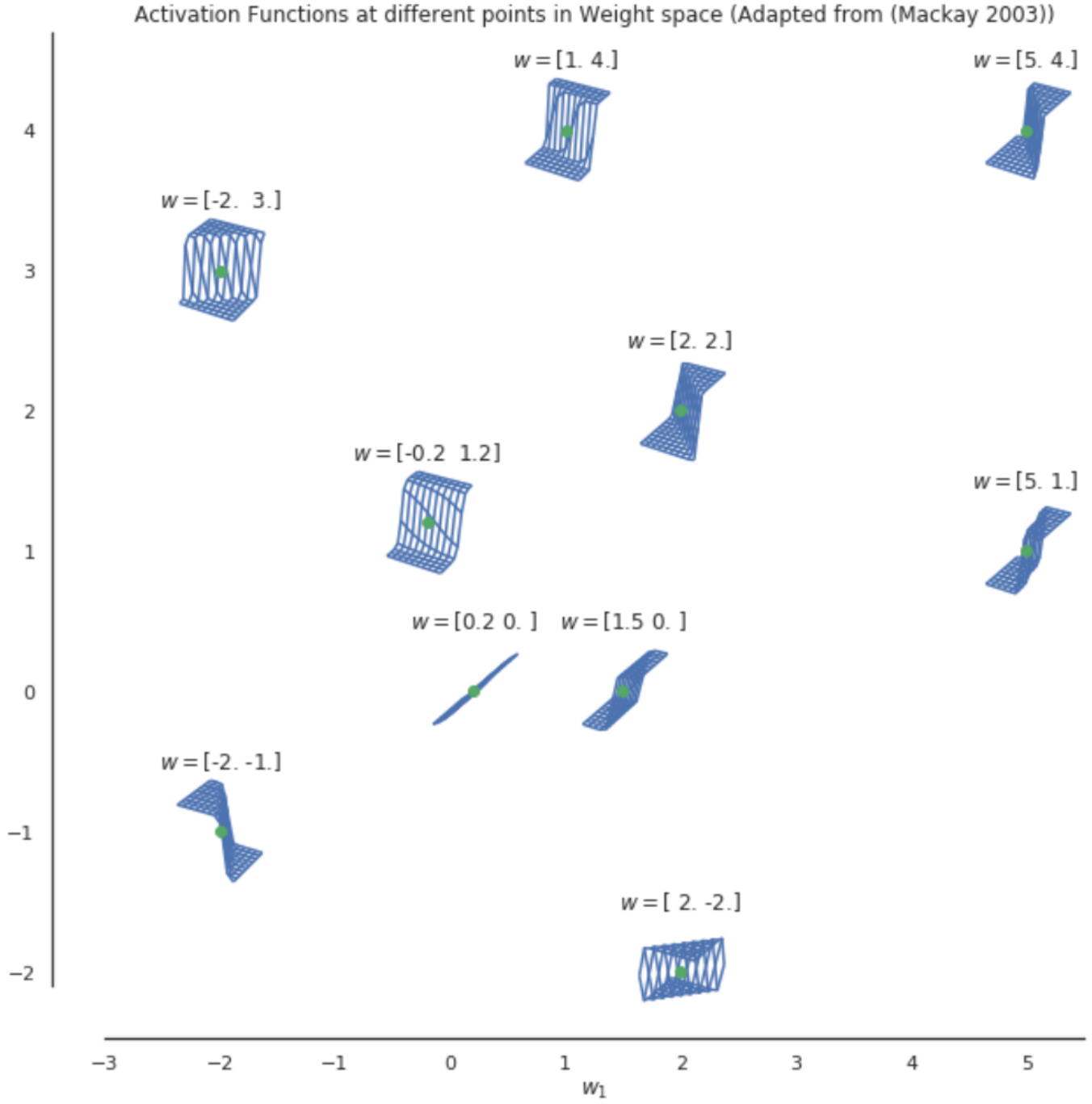
It squashes the neuron's output between 0 and 1, is always positive, is bounded, and is strictly increasing. Its derivative is $\frac{da}{dz} = a(1 - a)$.



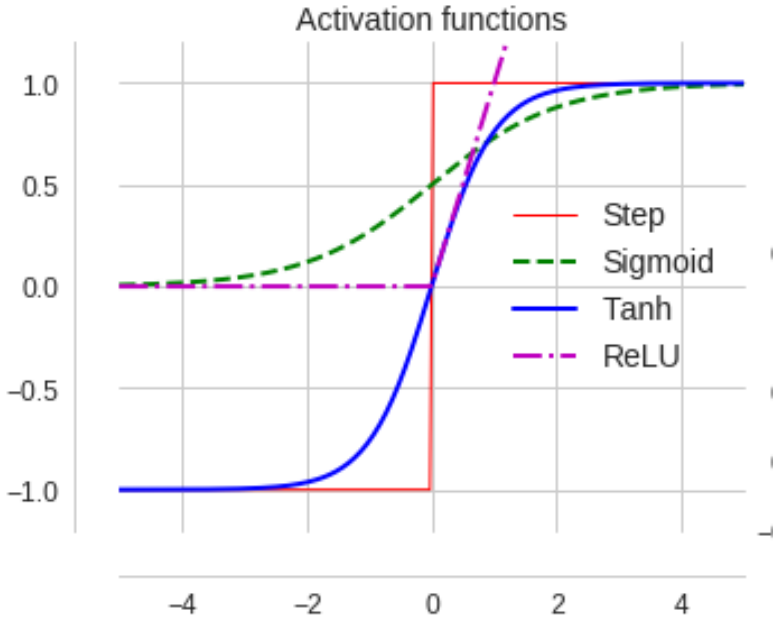
Sigmoids in multiple dimensions



Decision boundary can vary enormously depending on the weights



Activation functions - tanh



Hyperbolic tangent (tanh):

$$a = \frac{\sinh z}{\cosh z} = \frac{e^z - e^{-z}}{e^z + e^{-z}} = \frac{e^{2z} - 1}{e^{2z} + 1}$$

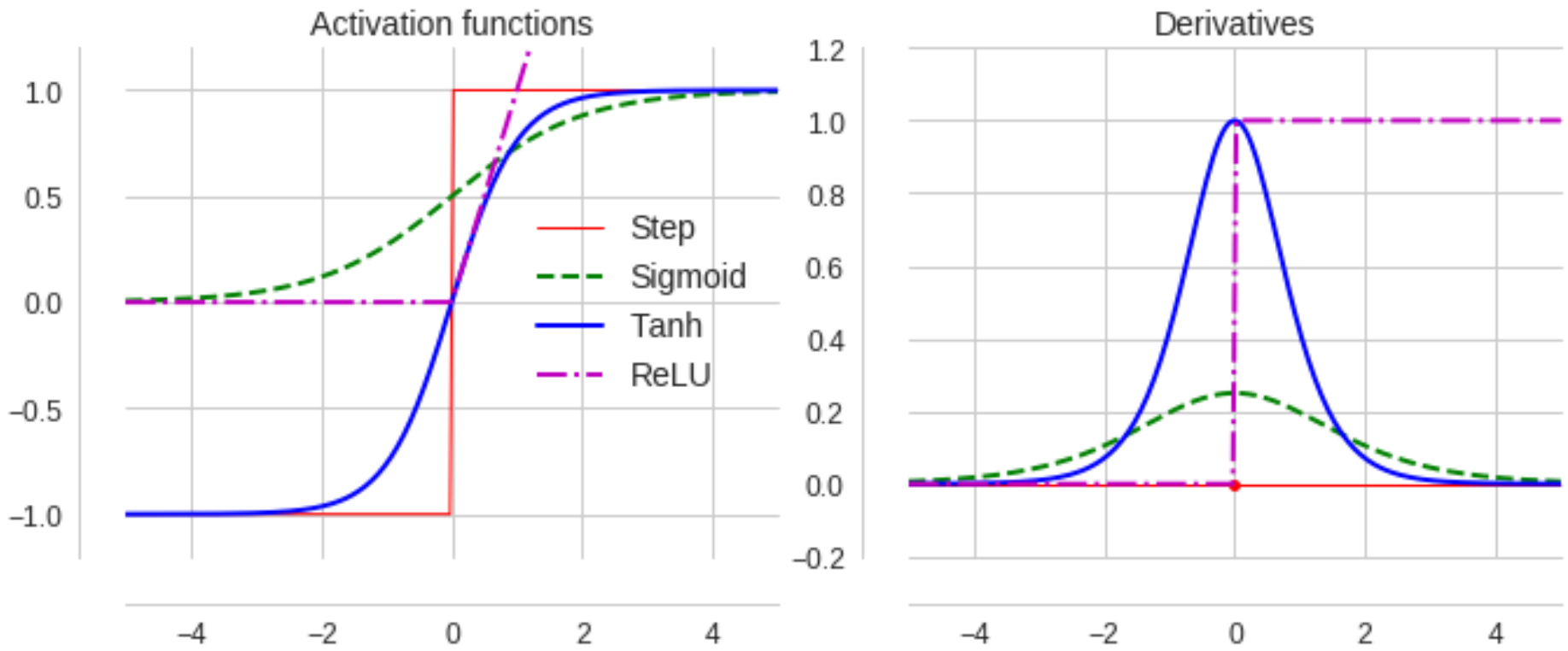
The hyperbolic tangent function is similar to the logistic/sigmoid function but its output range is from -1,1 instead of 0,1, which makes each layer more or less nromalized and which can speed up learning. Squashes the neuron's output between -1 and 1, so can be positive or negative. It is bounded and strictly increasing.

Its derivative is

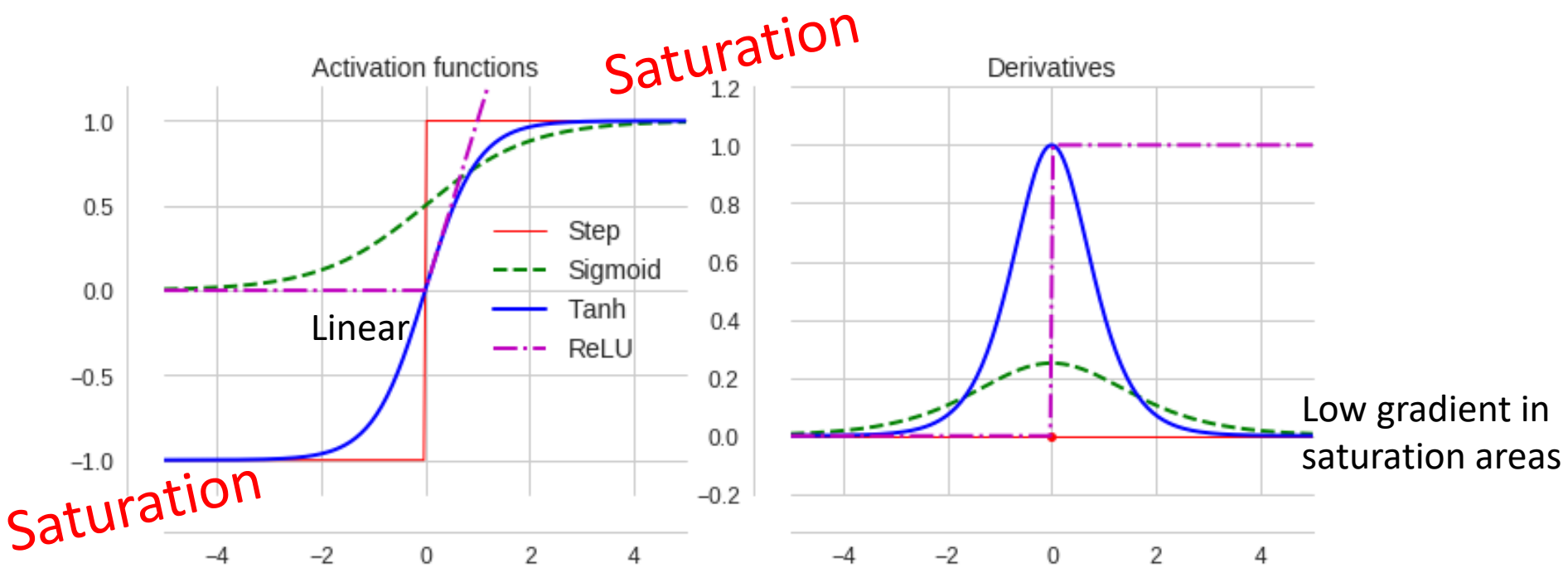
$$\frac{da}{dz} = (1 - a^2) = 1 - \tanh^2 z$$

.

Derivative of activation function



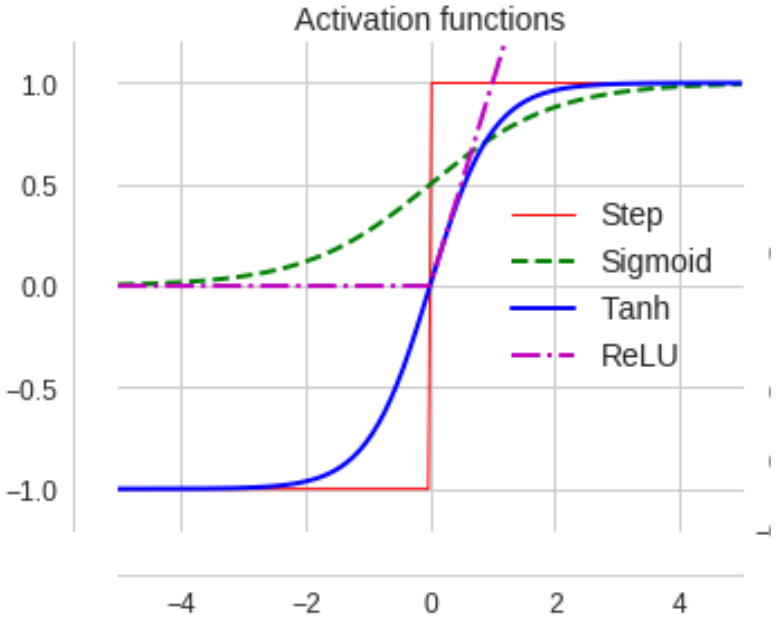
Saturation issues



Low gradient in saturation areas

See [(Glorot & Bengio 11)](#) for details

Activation functions ReLU



The **ReLU** was a later development designed to reduce the 'disappearing gradient' problem in multi layered networks. The mapping is

$$a = \max\{0, z\},$$

where z is usually the output of an affine transformation of inputs x such as $W^T x + b$,

$$a = \max\{0, W^T x + b\},$$

where W are the weights of the neuron from the previous layer's activations x, and b is an offset, or bias term. Bounded below by 0 (always non-negative), but is not upper bounded. It is strictly increasing. It tends to give neurons with sparse activities. Its derivative is 1 in regions where $z > 0$ and 0 elsewhere.

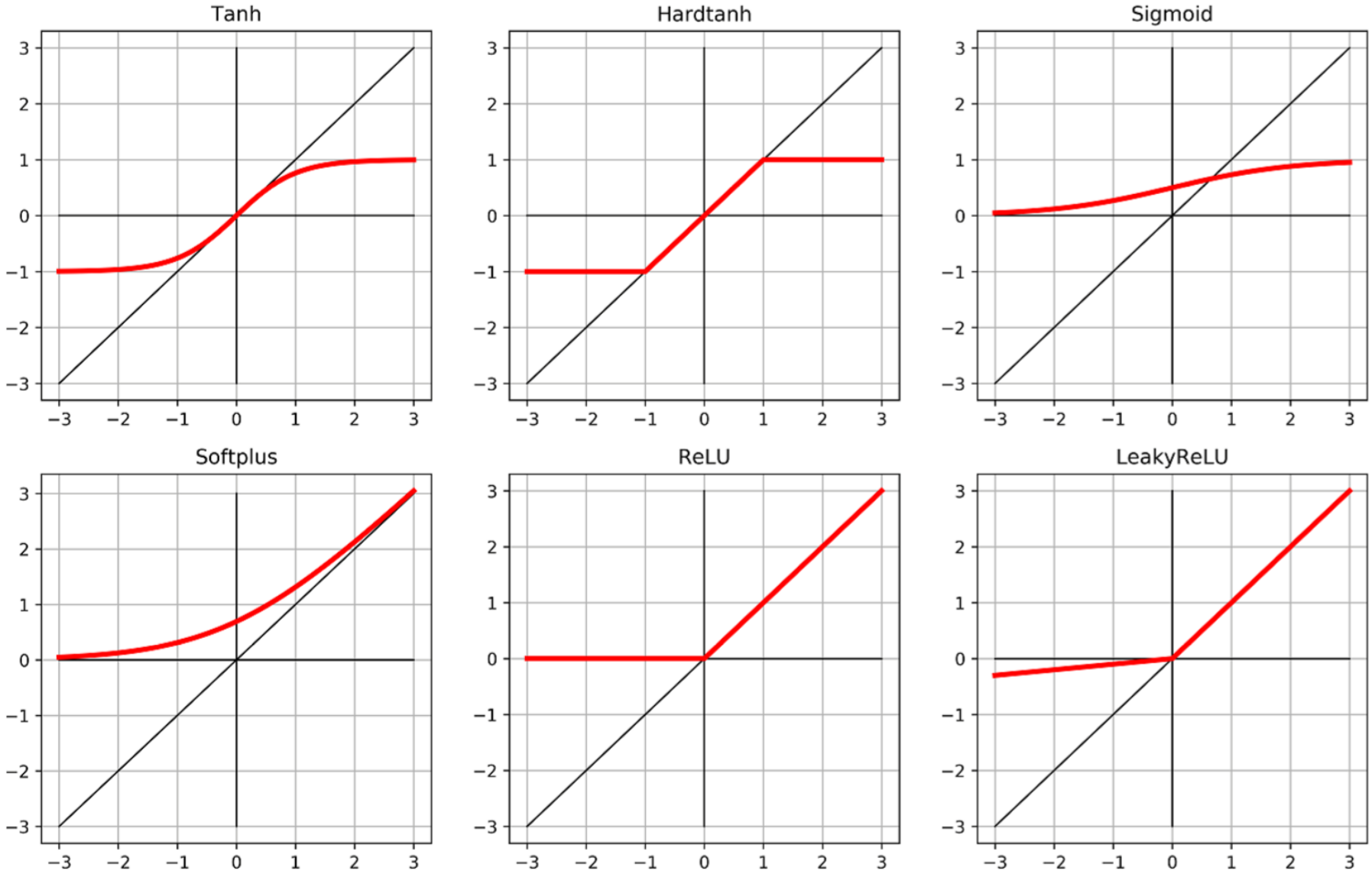


Figure 6.5 A collection of common and not-so-common activation functions.

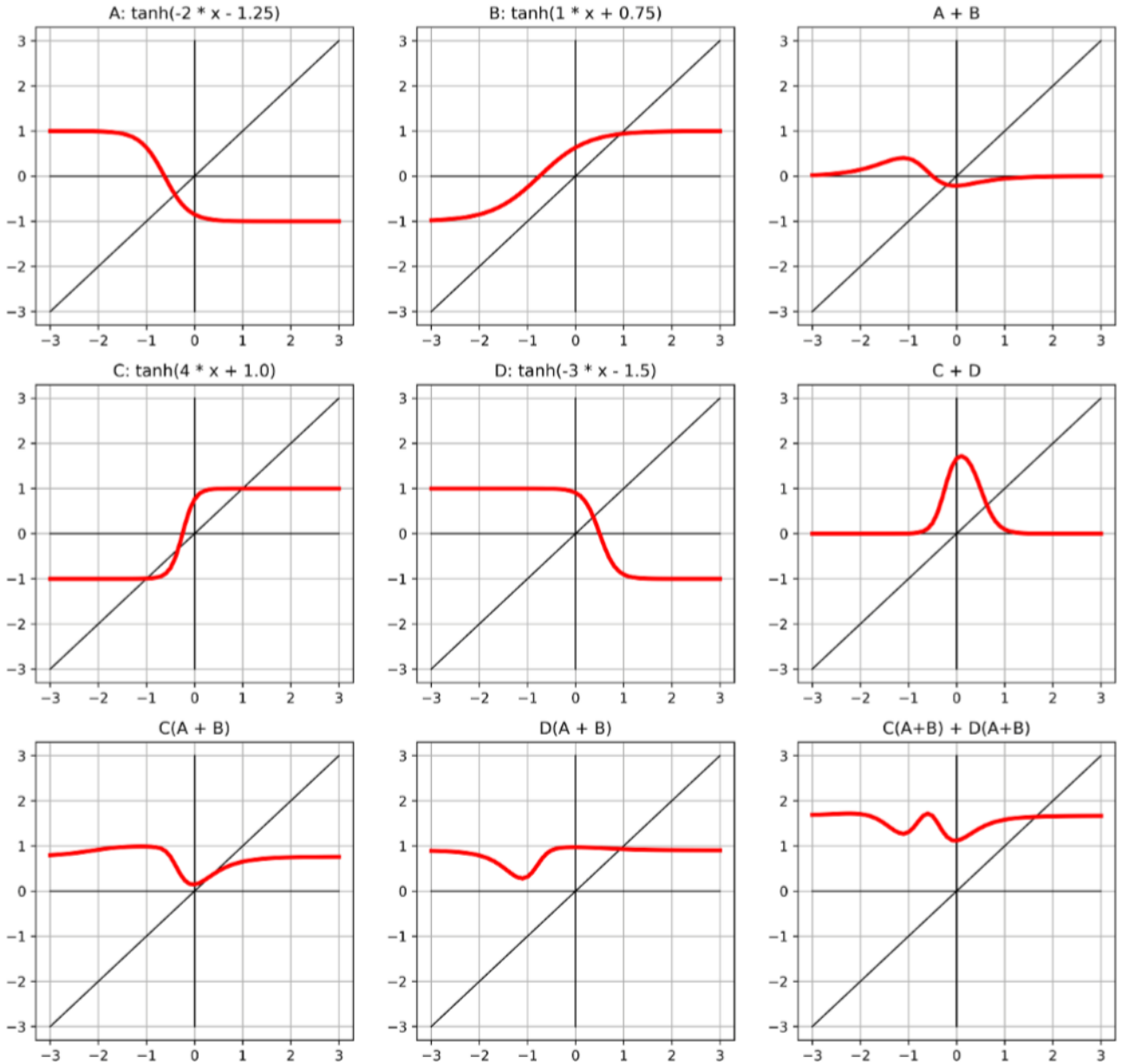


Figure 6.6 Composing multiple linear units and tanh activation functions to produce nonlinear outputs.

Summary

- You should have a clear idea of the architecture of a feed-forward neural network consisting of layers, neurons, weights, biases and activation functions.
- The network can be seen as acting like a non-linear function f(**x**; **w**) to predict a (potentially vector) value for a given (potentially vector) input **x**.
- You should understand how stochastic gradient decent can be used to optimize the weight and bias parameters in the network to give minimal loss when predicting training data.

- **READING: If you are uncertain about the maths involved in this lecture then it might be good to refresh this with Appendix A, B and C of Simon's book. Otherwise much more detail about this material can be found in Chapter 3 and 4 of Simon's book.**